Deep Learning

EEP 596 Computer Vision: Classical and Deep Methods (Autumn 2026)

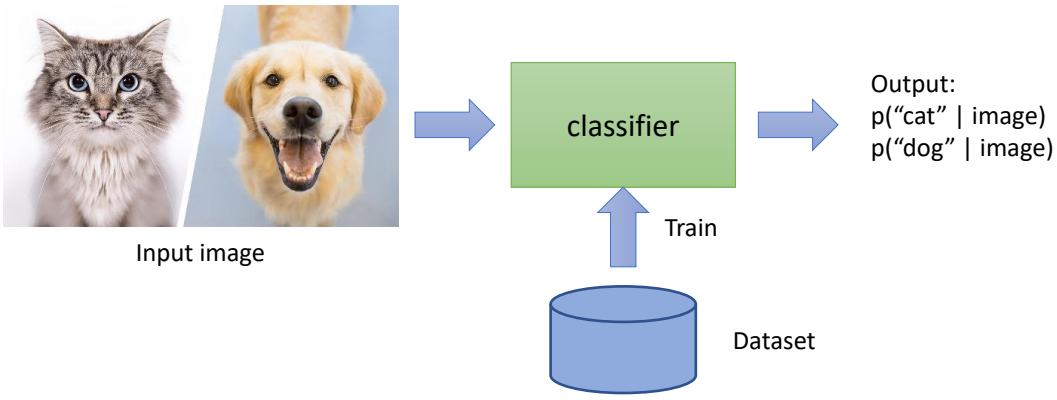
UNIVERSITY of WASHINGTON

Stan Birchfield

NVIDIA

1

Recall: Binary classification

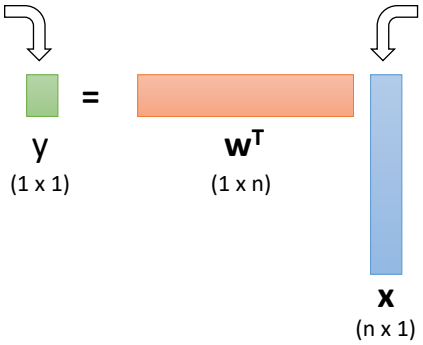


2

Let's try matrix multiplication

Output:
p("cat" | image)

= 1 – p("dog" | image)



Input:
The image pixel values stacked as a vector

$$y = \mathbf{w}^T \mathbf{x}$$

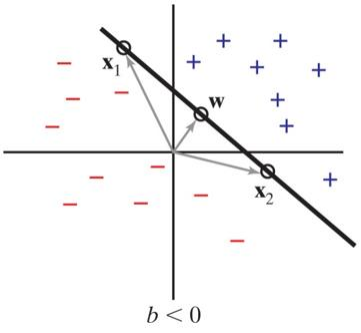
Weight matrix

This scalar output is the signed distance to the decision boundary

3

Linear discriminant function (1936)

$$g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b \qquad \|\mathbf{w}\| = 1$$



where

- $\mathbf{x}$ feature vector
- $\mathbf{w}$ weight vector (to be learned)
- b bias (to be learned)

- $\mathbf{w}$ defines separating hyperplane
- $\mathbf{w}$ is normal to hyperplane
- $|b|$ is distance from origin to hyperplane
- $g(\mathbf{x})$ is signed distance to hyperplane

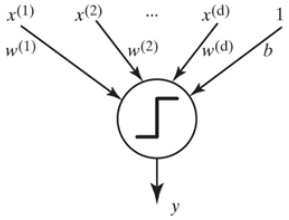
https://github.com/fastai/fastbook/blob/master/01_intro.ipynb

4

Perceptron

(1943)

Now we can **threshold** to make a decision



Perceptron is linear function followed by threshold

This is the simplest neural network

$$y = h(\mathbf{w}^\mathsf{T}\mathbf{x} + b)$$

h is Heaviside function:

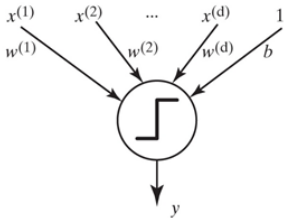
Can solve problems that are **linearly separable**

$$h(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

5

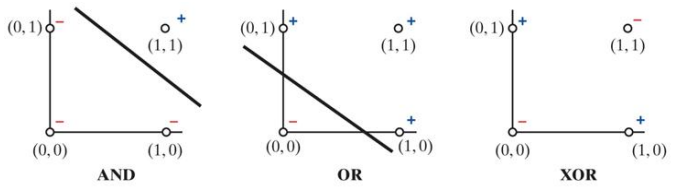
Linearly separable problems

A problem is linearly separable if classes are separated by a hyperplane



XOR is a simple counterexample (**not** linearly separable)

$$y = h(\mathbf{w}^\mathsf{T}\mathbf{x} + b)$$



6

Multilayer perceptron (1957)

Multiple perceptrons organized into layers

Each layer has a certain number of units

Each unit is a perceptron

This is

a 2-layer network

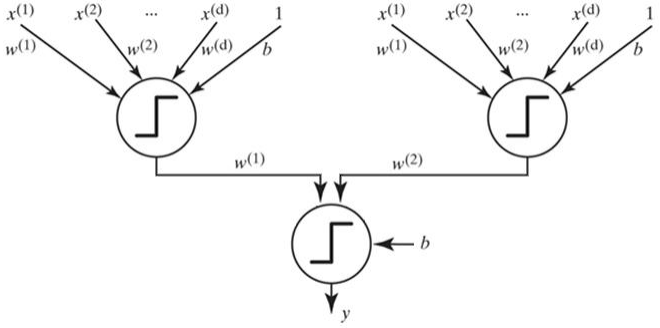
with

2 units in the first layer

and

1 unit in the final layer

(Weights are not necessarily equal)



More powerful than perceptron

7

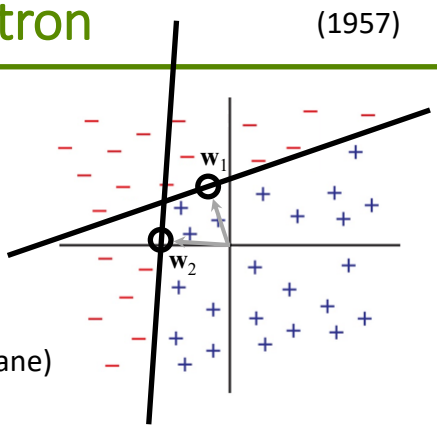
Multilayer perceptron (1957)

What if problem is not linearly separable?

Try two separate hyperplanes

$$g_1(\mathbf{x}) = \mathbf{w}_1^T\mathbf{x} + b_1$$

$$g_2(\mathbf{x}) = \mathbf{w}_2^T\mathbf{x} + b_2$$



How to combine?

Linear combination does not work (just another hyperplane)

$$g(\mathbf{x}) = \alpha g_1(\mathbf{x}) + \beta g_2(\mathbf{x})$$

Instead, use non-linear combination

$$g(\mathbf{x}) = (g_1(\mathbf{x}) < 0) \quad and \quad (g_2(\mathbf{x}) < 0)$$

Equivalently,

$$g(\mathbf{x}) = h(g_1(\mathbf{x})) + h(g_2(\mathbf{x}))$$

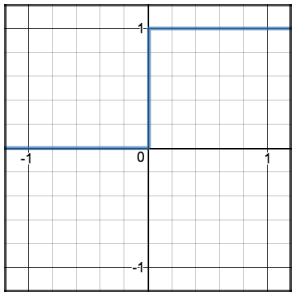
Heaviside function

$$h(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

8

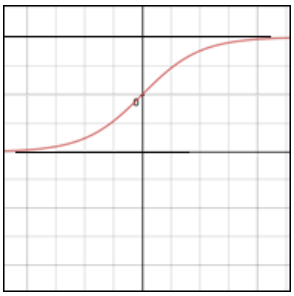
Neural network (1986)

- Difficult to train multilayer perceptron
- Breakthrough: Use differentiable **activation function** – backpropagation
- Now errors in output can relate to changes in input through derivatives



hard threshold (Heaviside)
(non-differentiable)

vs.

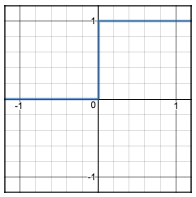


soft threshold
(differentiable)

9

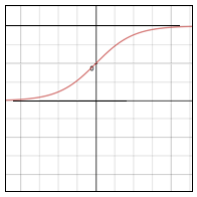
Activation functions

1960
Heaviside
(hard threshold)



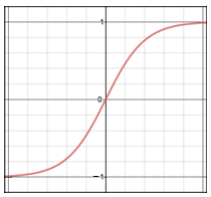
$$= \begin{cases} 1 & z \geq 0 \\ 0 & \text{o.w.} \end{cases}$$

1986
sigmoid
(soft threshold)



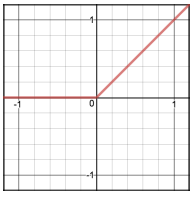
$$= \frac{1}{1+e^{-z}}$$

tanh



$$= \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$$

2010
ReLU
"rectified linear unit"



$$= \begin{cases} z & z \geq 0 \\ 0 & \text{o.w.} \end{cases}$$

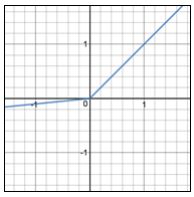
This should be your default

10

Activation functions (cont.)

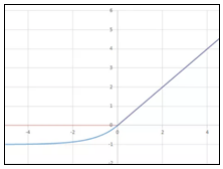
Pros and cons of each?

2014
leaky ReLU,
PReLU



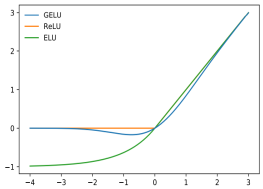
$$= \begin{cases} z & z \geq 0 \\ \alpha z & \text{o.w.} \end{cases}$$

2015
ELU

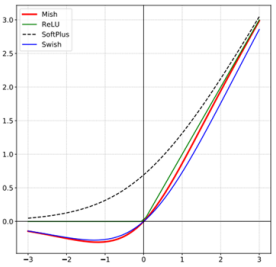


$$= \begin{cases} z & z \geq 0 \\ \alpha(e^z - 1) & \text{o.w.} \end{cases}$$

2016
GELU



2010 – 2019
SoftPlus, SELU,
Swish, Mish



https://ml-cheatsheet.readthedocs.io/en/latest/activation_functions.html

11

Activation functions (cont.)

"Gaussian error linear unit"

$$\text{GELU}(x) = xP(X \leq x) = x\Phi(x) = x \cdot \frac{1}{2}\left[1 + \text{erf}(x/\sqrt{2})\right]$$

Gaussian random variable $X \sim \mathcal{N}(0,1)$

Gaussian CDF

Gaussian error function $\text{erf}\, z = \frac{2}{\sqrt{\pi}} \int_0^z e^{-t^2}\, \text{d}t$

$$\approx 0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$$

$$\approx x\sigma(1.702x)$$

sigmoid

"Sigmoid linear unit"

SiLU $x\sigma(x)$

- SoftPlus
  f(x) = log(1 + exp(*x*))
  approximates ReLU
  [Nair, Hinton, 2010]
- SELU – scaled exponential

$$f(x) = \lambda \begin{cases} x & \text{if } x \geq 0 \\ \alpha(\exp(x) - 1) & \text{if } x < 0 \end{cases}$$

  [Klambauer et al., 2017]
- Swish
  f(x) = *x* sigmoid(β *x*)
- Mish
  f(x) = *x* tanh(softplus(*x*))

https://arxiv.org/abs/1606.08415
https://arxiv.org/pdf/1710.05941
https://arxiv.org/abs/1908.08681

12

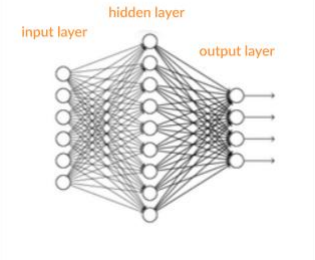
How many layers?

Theorem: Any problem can be solved by a 2-layer network
"universal function approximator"

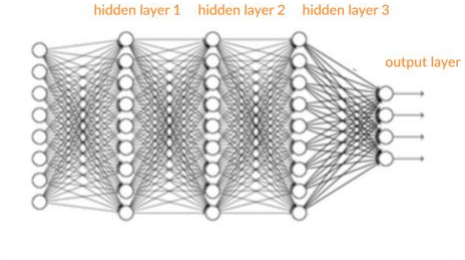
So why are more than 2 layers used in practice?

Empirical observation: Just seems to work better, easier to train

2-layer (shallow) network



"deep" network

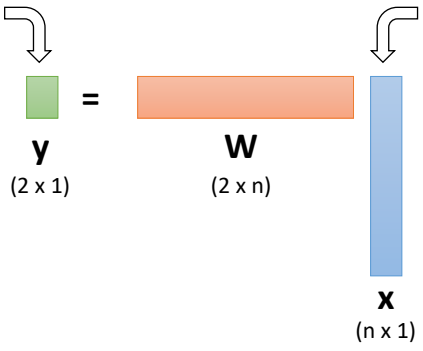


Note: "Input layer" is not really a layer. It does not do anything.

13

Let's revisit matrix multiplication

Output:
PDF over the classes
p("cat" | image)
p("dog" | image)



Input:
The image pixel values
stacked as a vector

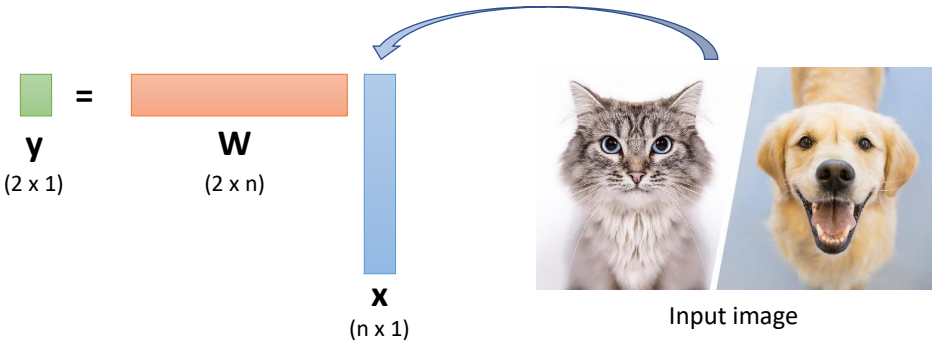
$$y = W x$$

Weight matrix

14

Deep learning

Output:
p("cat" | image)
p("dog" | image)



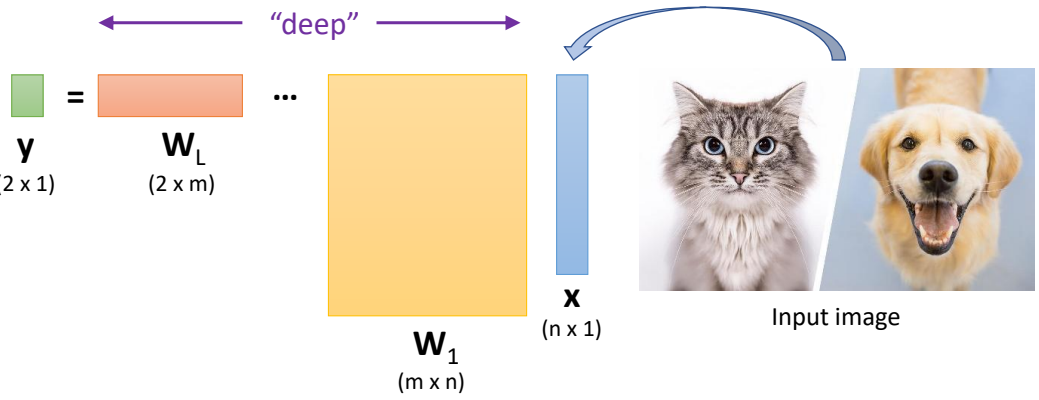
$$y = W x$$

Not enough parameters for such a complex problem…

15

Deep learning

Output:
p("cat" | image)
p("dog" | image)

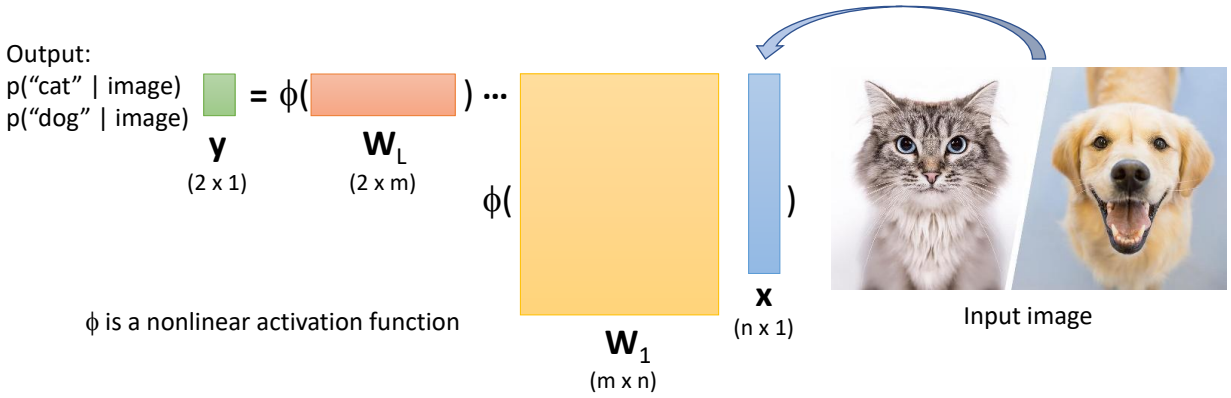


$$y = W_L (\cdots W_1 (x))$$

Linear solution – collapses to the previous…

16

Deep learning



ϕ is a nonlinear activation function

(Inference) Forward pass to compute output **y**

$$\mathbf{y} = \phi(\mathbf{W}_L \cdots \phi(\mathbf{W}_2\, \phi(\mathbf{W}_1\, \mathbf{x})))$$

(Training) Backward pass to compute weights **W**

Key idea: Replace manually crafted features with automatically learned features

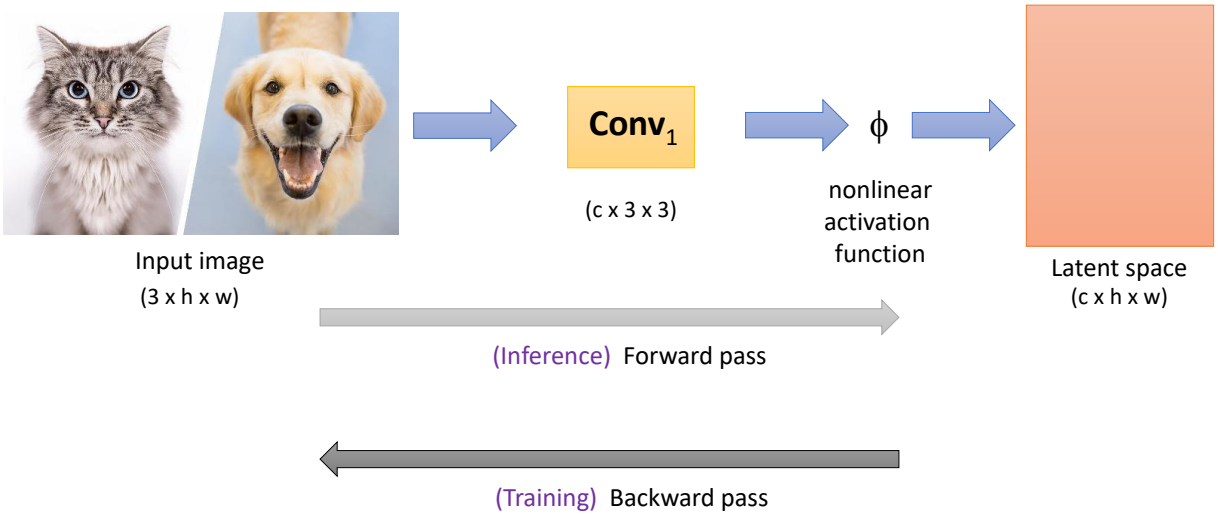
17

Projection



18

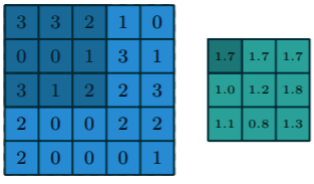
Deep learning with convolution



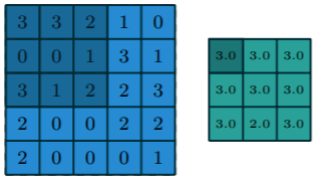
19

Pooling

- Pooling is used to reduce the size of the image
- Also involves sliding window
- Two types:
  - Average pooling (same as convolution)
  - Max pooling (nonlinear operation)



Average pooling
No padding
Stride = 1



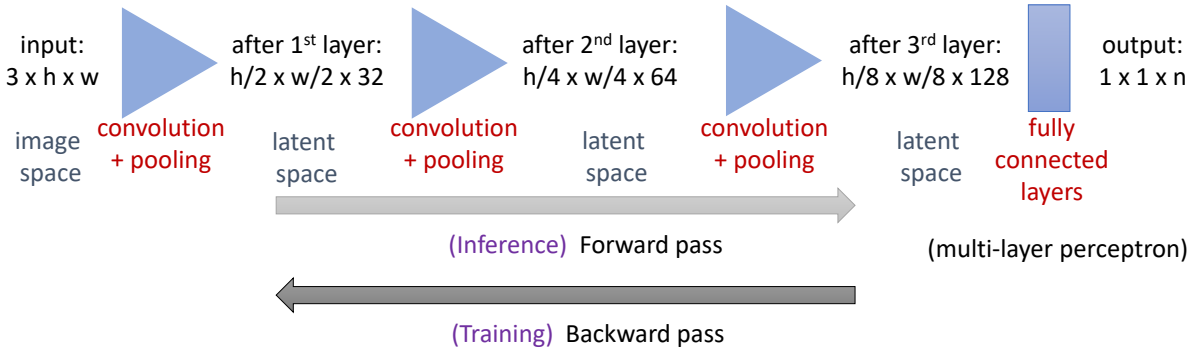
Max pooling
No padding
Stride = 1

V. Dumoulin, F. Visin, A guide to convolution arithmetic for deep learning, https://arxiv.org/pdf/1603.07285.pdf

20

Encoder architecture

- Encoder (or backbone) captures image information in *latent space*
- Rule of thumb:
  - Image gets smaller (÷ 2)
  - Number of feature channels get bigger (x 2)



21

Logits and Softmax

Sigmoid

Domain: Real

Range: [0,1]

Maps real number to probability

$$S(x) = \frac{1}{1+e^{-x}} = \frac{e^x}{e^x+1}$$

Logit

Domain: [0,1]

Range: Real

Maps probability to real number

Inverse of sigmoid

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

Activation of last layer is linear

Network outputs real numbers in last layer

Let's interpret these as "logits"

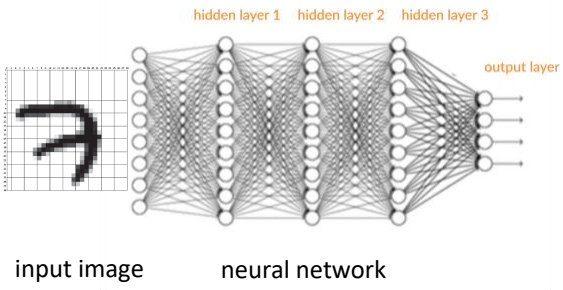
Softmax converts them to probabilities

$$\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Logit value	Softmax computation	Softmax outcome
2.0	$\frac{exp(x_i)}{\sum_j exp(x_j)} = \frac{exp(2.0)}{exp(2.0)+exp(4.3)+exp(1.2)+exp(-3.1)}$	0.087492
4.3	$\frac{exp(x_i)}{\sum_j exp(x_j)} = \frac{exp(4.3)}{exp(2.0)+exp(4.3)+exp(1.2)+exp(-3.1)}$	0.872661
1.2	$\frac{exp(x_i)}{\sum_j exp(x_j)} = \frac{exp(1.2)}{exp(2.0)+exp(4.3)+exp(1.2)+exp(-3.1)}$	0.039312
-3.1	$\frac{exp(x_i)}{\sum_j exp(x_j)} = \frac{exp(-3.1)}{exp(2.0)+exp(4.3)+exp(1.2)+exp(-3.1)}$	0.000292
Sum		0.999757
(rounded)		1

22

Logits and Softmax



input image neural network

MNIST dataset

"logit" output

2.0
4.3
1.2
-3.1

apply softmax

$$\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

PDF

0.087
0.873
0.039
0.000

N categories

$\Sigma = 1$

23

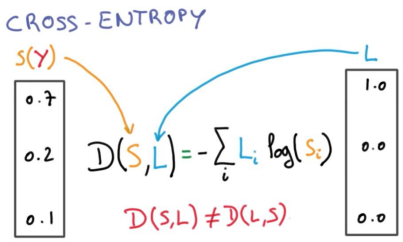
Calculating error / loss

How to measure error?

network outputs this:	ground truth one-hot vector:
0.087	0.0
0.873	0.0
0.039	0.0
0.000	1.0

Compare these two PDFs:

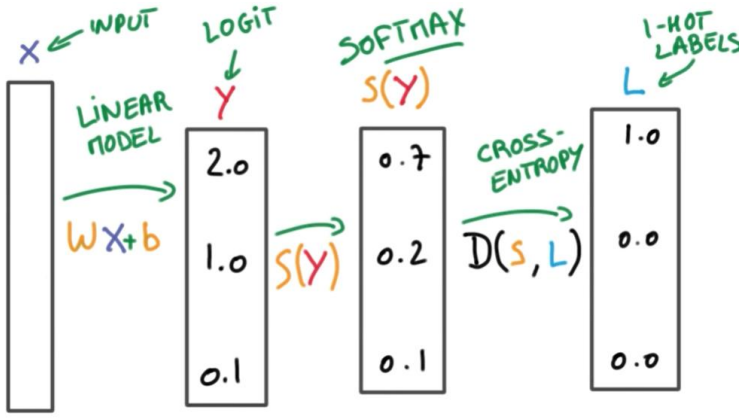
- L1 loss
- L2 loss
- cross-entropy loss



https://medium.com/data-science-bootcamp/understand-cross-entropy-loss-in-minutes-9fb263caee9a

24

Logits, softmax, cross-entropy loss



https://medium.com/data-science-bootcamp/understand-cross-entropy-loss-in-minutes-9fb263caee9a

25

Cross-entropy loss

Two distributions *p* and *q*

Difference b/w distributions can be measured by cross-entropy

Also called logistic loss, log loss (essentially same thing)

Related to Kullback-Leibler divergence

$$\begin{aligned} H(p,q) &= -\mathbb{E}_p[\log q] \\ &= -\sum_i p_i \log q_i \\ &= H(p) + D_{KL}(p\|q) \end{aligned}$$

```
# calculate cross entropy
def cross_entropy(p, q):
    return -sum([p[i]*log2(q[i]) for i in range(len(p))])
```

Binary cross-entropy

$$H(p,q) = -(y \log \hat{y} + (1-y) \log(1-\hat{y}))$$

26

Stochastic gradient descent (SGD)

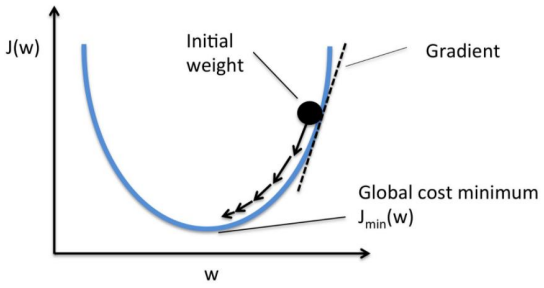
To train network, need to adjust weights based on loss

Pass data through network (forward pass), compute loss, then use backprop / chain rule

Update weights

$$w_i \leftarrow w_i - \alpha \cdot \frac{\partial \text{loss}}{\partial w_i}$$

where α is the learning rate



Why "stochastic"?

Cannot use all data at once (too big)

Split into batches

Perform gradient descent one batch at a time

Results will depend on order in which batches are encountered

27

Data augmentation

Manually labeling images is time-consuming, tedious, expensive

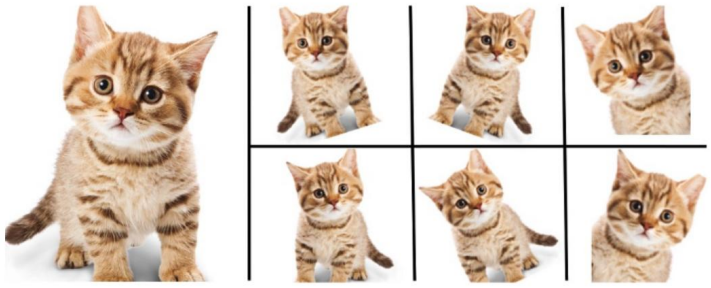
How to make the most use of our data?

Data augmentation – one labeled image can be used many times

Makes network more robust, b/c it sees more variations

Common:

- flip
- rotation (within plane)
- scale
- translation
- crop
- add Gaussian noise
- color shift



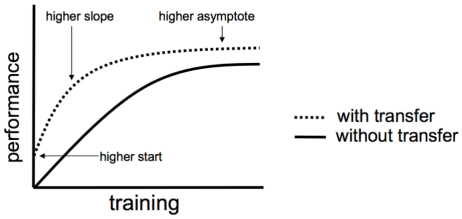
https://nanonets.com/blog/data-augmentation-how-to-use-deep-learning-when-you-have-limited-data-part-2/

28

Transfer learning

Typical network has millions/billions of parameters
Training a network takes a long time
How to speed up?
Reuse work of others
Key insight: Early layers of networks are task-independent
Solution: Copy weights learned elsewhere, fine-tune on your dataset
Called transfer learning
Will converge much faster
Use off-the-shelf architecture, trained on ImageNet:
- VGG
- Inception
- ResNet

Remove final layers, can freeze backbone to save time

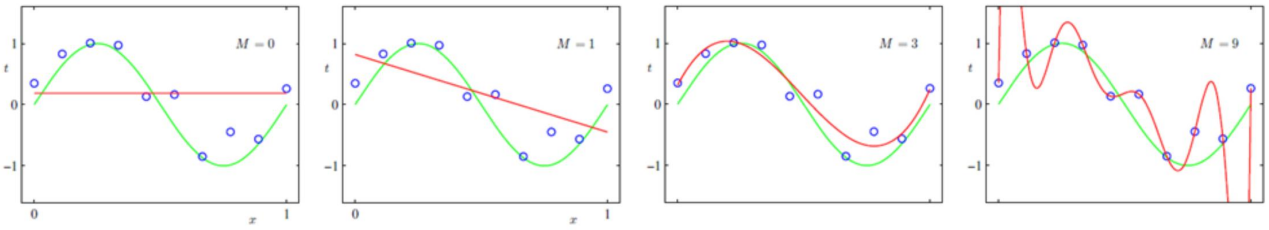


https://machinelearningmastery.com/transfer-learning-for-deep-learning/

29

Regularization

Fit polynomial to points. Higher-order polynomials fit training data better. Too high → overfitting.



Regularization is used to avoid overfitting.

Polynomial model: $y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M = \sum_{j=0}^{M} w_j x^j$

Loss function to be minimized: $\sum_{n=1}^{N} \left(y(x_n, \mathbf{w}) - t_n\right)^2 + \boxed{\lambda \|\mathbf{w}\|^2}$ L_2 regularization

where $\|\mathbf{w}\|^2 \equiv \mathbf{w}^{\mathsf{T}}\mathbf{w} = w_0^2 + w_1^2 + \ldots + w_M^2$

C. Bishop, *PR and ML*

30

PyTorch tutorials

- PyTorch with examples
  https://pytorch.org/tutorials/beginner/pytorch_with_examples.html
- PyTorch 60 minute blitz
  https://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html
- Demystifying Convolutions in PyTorch
  https://engineering.purdue.edu/DeepLearn/pdf-kak/week6.pdf

31